

An Ultra-Low-Power Synthesizable Asynchronous AER Encoder for Neuromorphic Edge Devices

Yihui Wang[†], Sheng-Yu Peng^{*}, Sahil Shah[†]

[†]University of Maryland, College Park, MD, USA

{yihuiw, sshah389}@umd.edu

^{*}National Yang Ming Chiao Tung University, Taiwan

speng@nycu.edu.tw

Abstract—This paper presents a fully synthesizable, tree-based Address-Event Representation (AER) encoder designed for scalable neuromorphic computing systems. To achieve high throughput while maintaining strict compatibility with commercial EDA workflows, the asynchronous design employs a bundled-data protocol within a semi-decoupled micropipeline. The architecture replaces traditional transparent latches with standard edge-triggered flip-flops, enabling digital synthesis and place-and-route (PnR) using Cadence toolkits. A cross-coupled NAND-based random-priority arbiter is embedded within the encoder of each tree node to resolve event collisions efficiently. An 8-event AER prototype is fabricated in 65 nm CMOS technology utilizing a purely digital standard-cell flow. Post-fabrication silicon measurements validate the design, demonstrating a peak throughput of 33 MEvent/s and an average event latency of 50 ns, equating to a propagation delay of 17 ns/(event-bit). The design consumes only 435 fJ per encoded event.

Index Terms—Asynchronous, Digital Circuit, Address-Event Representation (AER), Neuromorphic computing, Bundled-data.

I. INTRODUCTION

Neuromorphic computing offers a highly energy-efficient, brain-inspired paradigm for artificial intelligence by relying on sparse, event-driven spikes [1]. However, efficiently routing these events across millions of neurons poses a critical architectural challenge. While neural traffic is predominantly sparse, it frequently features dense, localized bursts, such as a dynamic vision sensor pixel jumping from millihertz background activity to kilohertz firing rates when stimulated [2], [3]. Traditional synchronous interconnects struggle with this unpredictable traffic, as constant global clock synchronization induces severe power overhead, latency bottlenecks, and network congestion.

To address this, the Address-Event Representation (AER) protocol time-multiplexes spikes into digital addresses over shared buses [4]–[6]. To further optimize these links, designers increasingly adopt the asynchronous bundled-data protocol [7]. In this approach, the global clock is replaced by localized handshaking circuits that bundle data lines with control signals. Memory elements, such as flip-flops (FFs) or latches, are triggered directly by asynchronous “request” and “acknowledge” signals rather than a continuous clock edge. By capturing data only when valid events are present and remaining idle otherwise, this event-driven mechanism naturally adapts to

neuromorphic traffic, sustaining high throughput with minimal active power [8].

Despite these benefits, integrating asynchronous logic into conventional synchronous design flows is difficult. Existing implementations often rely on labor-intensive, full-custom circuits with limited scalability [9], or utilize experimental, custom EDA tools that introduce steep learning curves [10]. To bridge this gap, this paper presents a fully synthesizable, tree-based asynchronous AER encoder implemented in 65 nm CMOS. To ensure seamless compatibility with commercial EDA workflows, we employ a four-phase bundled-data protocol. Unlike traditional micropipelines reliant on transparent latches [7], our architecture utilizes standard edge-triggered FFs within a semi-decoupled micropipeline, allowing for straightforward logic synthesis and place-and-route. Enhanced by embedded cross-coupled NAND-based random-priority arbiters for efficient collision resolution, post-fabrication silicon measurements validate the design, demonstrating a peak throughput of 33 MEvent/s and an exceptional energy efficiency of 435 fJ/event.

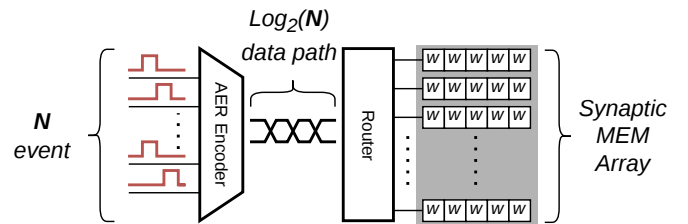


Fig. 1. The Address-Event Representation (AER) protocol. The AER system encodes events from the neuron array, and the resulting address is routed for downstream synaptic memory access.

II. PROPOSED ASYNCHRONOUS ARCHITECTURE

A. Address-Event Representation (AER) Protocol

To facilitate efficient, event-driven communication without the severe power overhead of global clocks, the neuromorphic SoCs employ localized asynchronous handshakes via the Address-Event Representation (AER) protocol [11]. Mimicking biological neural networks, discrete neural spikes are usually packetized into spatial digital addresses and routed directly to destination synaptic memories [12]. Fig. 1 shows the block diagram of the AER encoder and routing to synaptic

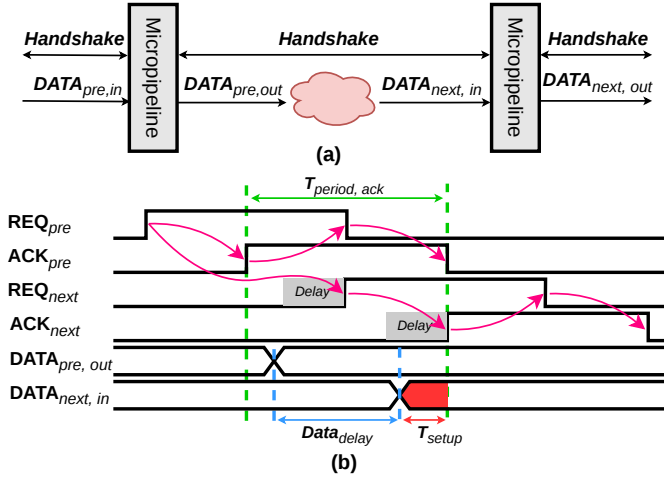


Fig. 2. The proposed asynchronous bundled-data micropipeline. (a) Datapath architecture detailing the routing of handshake and data signals between adjacent stages. (b) Time sequence diagram of the modified protocol. REQ_{pre} and ACK_{pre} interface with the previous stage; REQ_{next} and ACK_{next} interface with the next.

memory. Valid address data is transmitted using a localized Request (REQ) and Acknowledge (ACK) handshake (Fig. 2).

In this work, we propose an architecture that aggregates events using a hierarchical binary tree. As in Fig. 3, at each pipeline stage, the arbitrated encoder receives inputs from its two child node encoders at the previous stage. The encoder generates a single-bit address that serves as a multiplexer select signal, determining which address from the previous stage is concatenated with the new bit. For an N -event system, this recursive multiplexing and concatenation operation propagates through $\log_2(N)$ stages, ultimately yielding the complete $\log_2(N)$ -bit source address of the original event.

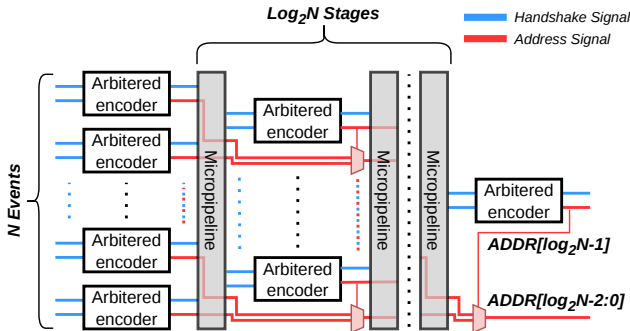


Fig. 3. The hierarchical AER tree structure. Blue lines represent asynchronous handshake signals, and red lines represent address data paths. At each pipeline stage, the accumulated address from the previous stage is multiplexed with the current stage's event address, merged, and forwarded.

B. 4-Phase Modified Semi-Decoupled Micropipeline Design

As illustrated in Fig. 3, to sustain high event throughput, we employ an asynchronous 4-phase bundled-data micropipeline. While 2-phase protocols can increase throughput by capturing data on both the rising and falling edges of a handshake signal,

they require specialized logic to trigger standard flip-flops (FFs)—such as the click-based protocol [13]—or rely on transparent latches for data registration [7]. These workarounds inherently introduce additional power and area overhead. In contrast, the 4-phase approach captures data exclusively on the rising edge, enabling the direct integration of standard FFs with minimal hardware overhead. It maintains high throughput by efficiently hiding the address multiplexing delay within the return-to-zero period of the handshake. To manage dense bursts of spikes and overcome the limitations of basic micropipelines [7]—which must wait for an acknowledgment (ACK) from the subsequent stage before acknowledging a preceding request—the design adopts a semi-decoupled architecture [14]. This architecture utilizes compact Earle latch-based C-elements as shown in Fig. 4(a), comprising just three AND gates and one OR gate [15]. By decoupling the input and output handshakes, this dual C-element configuration allows the pipeline to respond to incoming requests immediately, independent of the subsequent stage's readiness.

In the proposed architecture, FFs are clocked strictly on the rising edge of the locally generated ACK signal rather than on the incoming REQ, as shown in Fig. 2, and the circuit structure is shown in Fig. 4(b). This significantly relaxes setup time requirements by hiding combinational datapath delays behind the controller's return-to-zero phase. To prevent overrun hazards and ensure safe data capture, matched delay elements are explicitly inserted into the REQ and ACK lines by Verilog parameter to encompass worst-case logic delays between stages, as illustrated in Fig. 4(b) and analyzed in Section III.

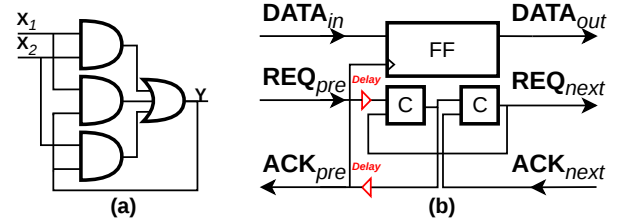


Fig. 4. (a) Earle latch-based C-elements. (b) Modified 4-phase Semi-Decoupled Micropipeline architecture using C-elements with flip-flop registers. The Delay is inserted to meet the timing constraint for the combination logic between pipeline stages, which is primarily a multiplexer in our AER design.

C. Stochastic Event Arbitration and Encoding

To resolve inevitable event collisions in tree-based AER architectures without the latency and state-tracking overhead of deterministic rotation priority schemes [16], we adopt a compact Multiplexed Arbitration design based on cross-coupled NAND gates [17] (Fig. 5). During asynchronous arrivals, the faster signal drives its respective NAND output low, locking out the competing channel while asserting REQ_{next} and the encoded address (ADDR). The losing request is held safely pending until the downstream node clears the transaction via ACK_{next} .

When competing events arrive near-simultaneously, the cross-coupled gates enter a metastable state, relying on inherent silicon mismatch and thermal noise to eventually break symmetry and randomly declare a winner. Rather than a hardware flaw, this stochastic tie-breaking mechanism mirrors biological neural networks, which harness cellular thermodynamic noise to prevent systemic deadlocks and enable flexible decision-making [18]. Thus, the asynchronous arbiter leverages physical noise for biologically plausible conflict resolution while preserving the temporal sparsity of neural activity. In this work the arbiter has been implemented at the input of the AER on the FPGA.

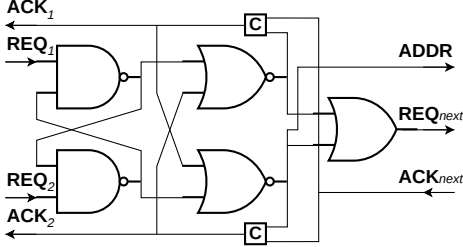


Fig. 5. Multiplexed Arbitration circuit. The inherent metastability of the cross-coupled NAND gates provides stochastic resolution for simultaneous event collisions.

III. CIRCUIT IMPLEMENTATION

A. Standard-Cell Circuit Implementation Flow

The proposed architecture is implemented using a fully automated digital RTL-to-GDSII flow. The parameterized AER module is first modeled in Verilog, beginning with the foundational Muller C-element [15], which forms the core of the micropipeline (Fig. 4) and the arbitrated encoder (Fig. 5). These components are hierarchically instantiated to construct the complete AER routing tree (Fig. 3), utilizing only standard digital logic gates.

The Verilog implementation is highly parameterized, allowing the number of input events, the number of pipeline stages, and the address bitwidths to be scaled according to application requirements. For this work, an 8-event AER template was implemented, fabricated, and characterized; the silicon measurement results are detailed in Section IV.

The physical implementation leverages the Cadence EDA toolkit: Genus for logic synthesis, Innovus for place-and-route (PnR), and Virtuoso for final padframe integration and global routing. Behavioral, post-synthesis, and post-layout simulations were conducted using Xcelium, utilizing Standard Delay Format (SDF) back-annotation at each stage to ensure timing accuracy. However, due to the absence of a global clock in asynchronous designs, standard Static Timing Analysis (STA) tools often struggle to verify these circuits seamlessly. To address this, an automated timing closure methodology was developed and is detailed in Section III-B.

B. Delay Matching and Timing Constraints

Rigorous timing analysis is critical in bundled-data pipelines to prevent metastable states from propagating. Previous works

[8], [16], [19] achieve timing closure using the *set_min_delay* command in Synopsys tools to constrain the handshake cycle. However, many synthesis engines, including Cadence Genus, lack equivalent support for asynchronous loops. To overcome this, we developed a tool-agnostic, automated methodology utilizing standard SDC constraints and iterative simulation.

As illustrated in Fig. 2(b), the ACK signals act as local clocks. Data launched at the current stage ($DATA_{pre,out}$) on the rising edge of ACK_{pre} must propagate through the combinational multiplexing logic and stabilize at the next stage ($DATA_{next,in}$) before the *subsequent* rising edge of ACK_{pre} . Therefore, the datapath delay must strictly satisfy: $Data_{delay} < T_{period,ack} - T_{setup}$.

To automatically satisfy this constraint across the entire AER without relying on custom EDA features, we employ a shell-scripted iterative methodology:

- 1) **Initial Synthesis:** Apply an initial *set_max_delay* SDC constraint to the datapath and synthesize the gate-level netlist with a baseline number of control path delay buffers.
- 2) **Timing Extraction:** Perform post-synthesis simulation with Standard Delay Format (SDF) back-annotation to extract the actual handshake cycle duration ($T_{period,ack}$) for each stage.
- 3) **Iterative Optimization:** Dynamically update the SDC *set_max_delay* constraint to be strictly less than 20% of the extracted $T_{period,ack}$ to ensure robust setup margins. If the synthesis Static Timing Analysis (STA) fails this new constraint, incrementally add delay buffers to the control path to extend $T_{period,ack}$ and repeat Steps 2 and 3 until the design is hazard-free.
- 4) **Physical Closure:** Execute Place and Route (PnR). If interconnect parasitics introduce new post-route datapath violations, increment the control delay buffers and re-verify.

By combining standard SDC bounds with automated log-parsing and simulation loops, this methodology provides a generic, highly reliable timing closure solution for standard-cell asynchronous designs utilizing edge-triggered flip-flops.

IV. MEASUREMENT RESULTS

A. Test Setup and Methodology

A prototype 8-event AER was fabricated using a 65 nm CMOS process, with the physical die details illustrated in Fig. 6. The stochastic arbiters were implemented exclusively at the input stage on the FPGA to verify their functionality. To evaluate the physical performance of the asynchronous router, the test chip was integrated into a breadboard-based test setup. A Xilinx Spartan-7 FPGA was utilized to inject high-speed, full-scan event streams into the test chip, ensuring the chip is working at full load.

The encoded output addresses and corresponding handshake signals are continuously routed back to the FPGA for real-time functional verification. Concurrently, a 100 MHz Digilent Analog Discovery logic analyzer monitors the asynchronous

control lines and output address data to validate routing correctness, assess pipeline performance, and ensure strict adherence to the bundled-data protocol. Latency and power measurements are averaged across all events captured within a 1 s measurement window. To quantify the system's energy efficiency, dynamic power consumption is calculated by measuring the voltage drop across a precision shunt resistor placed in series with the 1.2 V core supply rail (V_{DD}).

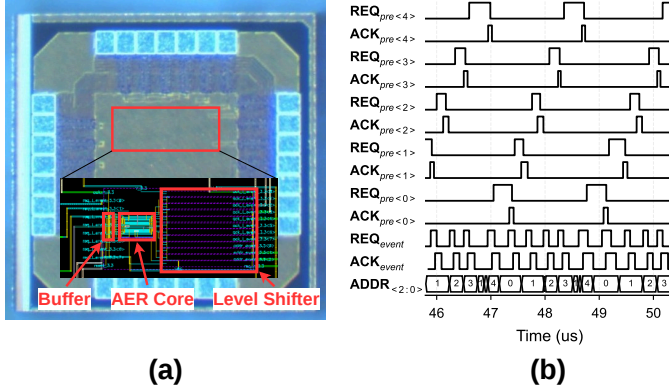


Fig. 6. (a) Die micrograph and layout details of the fabricated AER encoder in 65 nm CMOS. The active asynchronous core measures $65.57 \mu\text{m} \times 44.6 \mu\text{m}$. (b) Selected measurement waveform slot of the testbench. The REQ_{pre} and ACK_{pre} are the input stage signals on the chip; the $\text{REQ}_{\text{event}}$, $\text{ACK}_{\text{event}}$, and ADDR are the output stage signals.

B. Performance Measurement and Analysis

In a tree-based micropipeline architecture, the overall system throughput is inherently bounded by the handshake completion rate of the initial pipeline stage, as depicted at the input of Fig. 3. Because the first stage is ready to process a subsequent event immediately after passing the current event forward, the maximum throughput is governed by the duration of a single handshake cycle (Fig. 2). During silicon validation, a 100 MHz clocked FPGA was utilized to inject full-scan events into the fabricated chip. Measurements (Fig. 6(b)) indicate an average handshake cycle duration of 30 ns between the FPGA and the asynchronous core, yielding a sustained peak throughput of ≈ 33 MEvent/s.

System latency, measured from the rising edge of the input request (REQ) signal to the falling edge of the output REQ signal for a single event, averages 50 ns across the tree. Normalizing this delay across the pipeline depth, which is 3 stages in our prototype, provides an average propagation latency of ≈ 17 ns/(event-bit).

Because the 1.2 V supply rail is shared with a co-fabricated analog design on the same die, the total power consumption was determined by combining measured dynamic power with simulated static power. The dynamic power during event processing was measured at $4 \mu\text{W}$ utilizing the voltage drop across a precision series shunt resistor. This active measurement was carefully isolated from the aggregate baseline leakage originating from the I/O pads and the shared analog circuitry. Meanwhile, the static power dissipation of the dedicated asynchronous AER core was estimated via simulation at

$4.7 \mu\text{W}$, yielding a total power consumption of $8.7 \mu\text{W}$. Based on this power profile, the proposed asynchronous encoder achieves an exceptionally high energy efficiency of 435 fJ per encoded event. For the 3-bit prototype design, this equates to an ultra-low 145 fJ/(event-bit).

C. Performance Summary and Architecture Comparison

Table I compares the proposed 65 nm AER encoder with recent asynchronous architectures [8]–[10]. The design achieves an exceptional measured energy efficiency of 145 fJ/(event-bit), outperforming both the simulated custom-EDA FP-AER (102.33 pJ/(event-bit)) [10] and the full-custom 28 nm Bi-AER (420 fJ/(event-bit)) [9]. While our 33 MEvent/s measured throughput exceeds the Bi-AER, it is theoretically outperformed by the simulated mask-based topology of Ouyang et al. (≈ 600 MEvent/s) [8]. Similarly, the Bi-AER leverages full-custom 28 nm transistor optimization to reach a fundamentally different latency regime (0.19 ns/(event-bit)) compared to the proposed 16.67 ns/(event-bit) [9]. However, this deliberate trade-off for automated standard-cell synthesizability does not bottleneck practical systems. State-of-the-art CMOS artificial sensory receptors peak at spike rates of only 1 kHz [20], and dense dynamic vision sensor bursts operate at the microsecond scale [3]. Thus, our 50 ns total latency and 33 MEvent/s bandwidth outpace peak biological and artificial requirements by orders of magnitude, proving that superior energy efficiency and sufficient routing speeds are achievable entirely within standard commercial EDA workflows.

TABLE I
MEASURED PERFORMANCE AND ARCHITECTURE COMPARISON

Metric	This Work	Purohit [10]	Ouyang [8]	Bi-AER [9]
Architecture	Binary Tree	Mixed Tree	Dual-Level Mask	Shared Bus
Implementation	65nm CMOS Standard Cell	65nm CMOS Custom EDA	65nm CMOS Standard Cell	28nm CMOS Custom Circuit
Synchronization	Asynchronous	Asynchronous	Asynchronous	Asynchronous
Throughput	33 MEvent/s	N/A	≈ 600 MEvent/s*	28.6 MEvent/s
Latency / Event-Bit	16.67 ns	20.23 ns*	49.88 ns*	0.19 ns
Energy / Event-Bit	145 fJ	102.33 pJ*	N/A	420 fJ

* Simulated results; not verified in silicon.

V. CONCLUSION

This paper presents a fully synthesizable, tree-based asynchronous AER encoder that integrates seamlessly into standard digital workflows. Silicon measurements from our 65 nm CMOS prototype validate the architecture's efficiency, demonstrating an average throughput of 33 MEvent/s, a 17 ns/(event-bit) propagation latency, and an ultra-low energy footprint of 435 fJ/event. This exceptional energy efficiency makes the design highly suitable for power-constrained neuromorphic edge devices, such as brain-machine interfaces and real-time neural decoders. Additionally, its modular and parameterized architecture ensures ready scalability to meet the massive event-routing demands of extremely large-scale neuromorphic systems.

REFERENCES

- [1] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, J. Hays, C. Young, C. Bartolozzi, A. Majumdar, S. G. Cardwell, M. Payvand, S. Buckley, S. Kulkarni, H. A. Gonzalez, G. Cauwenberghs, C. S. Thakur, A. Subramoney, and S. Furber, "Neuromorphic computing at scale," *Nature*, vol. 637, no. 8047, pp. 801–812, Jan. 2025.
- [2] B. Huo, F. Li, S. Peng, H. Chen, S. Xin, and H. Wang, "Research on SNN Learning Algorithms and Networks Based on Biological Plausibility," *IEEE Access*, vol. 13, pp. 95 243–95 256, 2025.
- [3] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128×128 120 dB 15 μ s Latency Asynchronous Temporal Contrast Vision Sensor," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Feb. 2008.
- [4] J. Park, T. Yu, S. Joshi, C. Maier, and G. Cauwenberghs, "Hierarchical Address Event Routing for Reconfigurable Large-Scale Neuromorphic Systems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2408–2422, Oct. 2017.
- [5] P. Purohit and R. Manohar, "Hierarchical Token Rings for Address-Event Encoding," in *2021 27th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, Sep. 2021, pp. 9–16, iSSN: 2643-1483.
- [6] C. Zamarreno-Ramos, A. Linares-Barranco, T. Serrano-Gotarredona, and B. Linares-Barranco, "Multicasting Mesh AER: A Scalable Assembly Approach for Reconfigurable Neuromorphic Structured AER Systems. Application to ConvNets," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 7, no. 1, pp. 82–102, Feb. 2013.
- [7] I. E. Sutherland, "Micropipelines," *Commun. ACM*, vol. 32, no. 6, pp. 720–738, Jun. 1989.
- [8] S. Ouyang, K. Zhou, H. Jiang, C. Li, J. Liang, F. Zhu, X. Zhang, and Q. Liu, "A Scalable Area-Efficient Low-Delay Asynchronous AER Circuits Design for Neuromorphic Chips," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 5, pp. 2804–2808, May 2024.
- [9] N. Qiao and G. Indiveri, "A bi-directional Address-Event transceiver block for low-latency inter-chip communication in neuromorphic systems," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2018, pp. 1–5, iSSN: 2379-447X.
- [10] P. Purohit and R. Manohar, "Field-programmable encoding for address-event representation," *Frontiers in Neuroscience*, vol. 16, Dec. 2022.
- [11] Y. Zhong, Y. Kuang, K. Liu, Z. Wang, S. Feng, G. Chen, Y. Yang, X. Cui, Q. Wang, J. Cao, S. Jia, Y. Liang, G. Sun, X. Cui, R. Huang, and Y. Wang, "PAICORE: A 1.9-Million-Neuron 5.181-TSOPS/W Digital Neuromorphic Processor With Unified SNN-ANN and On-Chip Learning Paradigm," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 2, pp. 651–671, Feb. 2025.
- [12] C. Frenkel, J.-D. Legat, and D. Bol, "MorphIC: A 65-nm 738k-Synapse/mm² Quad-Core Binary-Weight Digital Neuromorphic Processor With Stochastic Spike-Driven Online Learning," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 13, no. 5, pp. 999–1010, Oct. 2019.
- [13] H. Wu, Z. Su, J. Zhang, S. Wei, Z. Wang, and H. Chen, "A Design Flow for Click-Based Asynchronous Circuits Design With Conventional EDA Tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 11, pp. 2421–2425, Nov. 2021.
- [14] S. Furber and P. Day, "Four-phase micropipeline latch control circuits," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 4, no. 2, pp. 247–253, Jun. 1996.
- [15] J. P. Murphy, "Design of latch-based C-element," *Electronics Letters*, vol. 48, no. 19, pp. 1–2, Sep. 2012, num Pages: 2.
- [16] J. Wei, J. Zhang, X. Zhang, Z. Wu, C. Dou, T. Shi, H. Chen, and Q. Liu, "An Asynchronous AER Circuits with Rotation Priority Tree Arbiter for Neuromorphic Hardware with Analog Neuron," in *2019 IEEE 13th International Conference on ASIC (ASICON)*, Oct. 2019, pp. 1–4, iSSN: 2162-755X.
- [17] A. Martin and M. Nystrom, "Asynchronous Techniques for System-on-Chip Design," *Proceedings of the IEEE*, vol. 94, no. 6, pp. 1089–1120, Jun. 2006.
- [18] H. A. Braun, "Stochasticity Versus Determinacy in Neurobiology: From Ion Channels to the Question of the "Free Will"," *Frontiers in Systems Neuroscience*, vol. 15, p. 629436, May 2021.
- [19] M. Gibiluka, M. T. Moreira, and N. L. Vilar Calazans, "A Bundled-Data Asynchronous Circuit Synthesis Flow Using a Commercial EDA Framework," in *2015 Euromicro Conference on Digital System Design*, Aug. 2015, pp. 79–86.
- [20] S. Subbulakshmi Radhakrishnan, A. Sebastian, A. Oberoi, S. Das, and S. Das, "A biomimetic neural encoder for spiking neural network," *Nature Communications*, vol. 12, no. 1, p. 2143, Apr. 2021.